


JURUSAN TEKNOLOGI INFORMASI

Mata Kuliah Pemrograman Mobile

# 07. Dasar Widget

Yoppy Yunhasnawa, S.ST., M.Sc.

# Topik

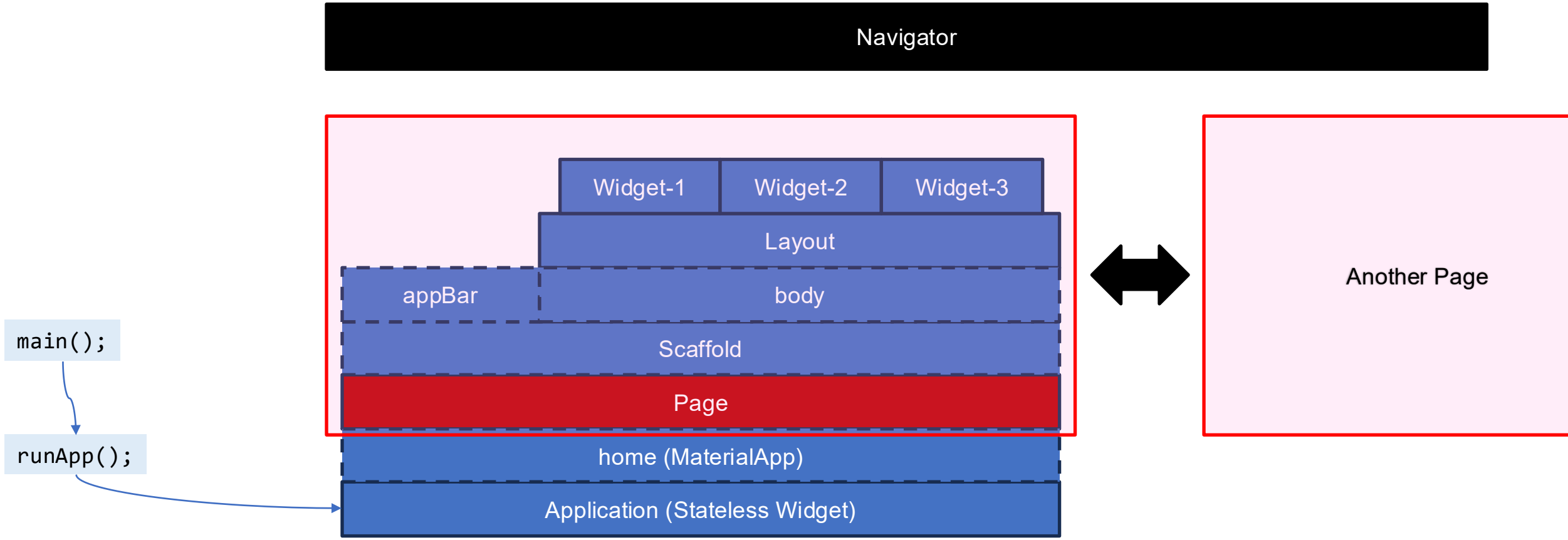
1. Widget Stack Dasar
2. Layout
3. Assets
4. Navigation
5. Tampilan Awal Aplikasi MyDJ

# *Topik-1:* Widget Stack Dasar

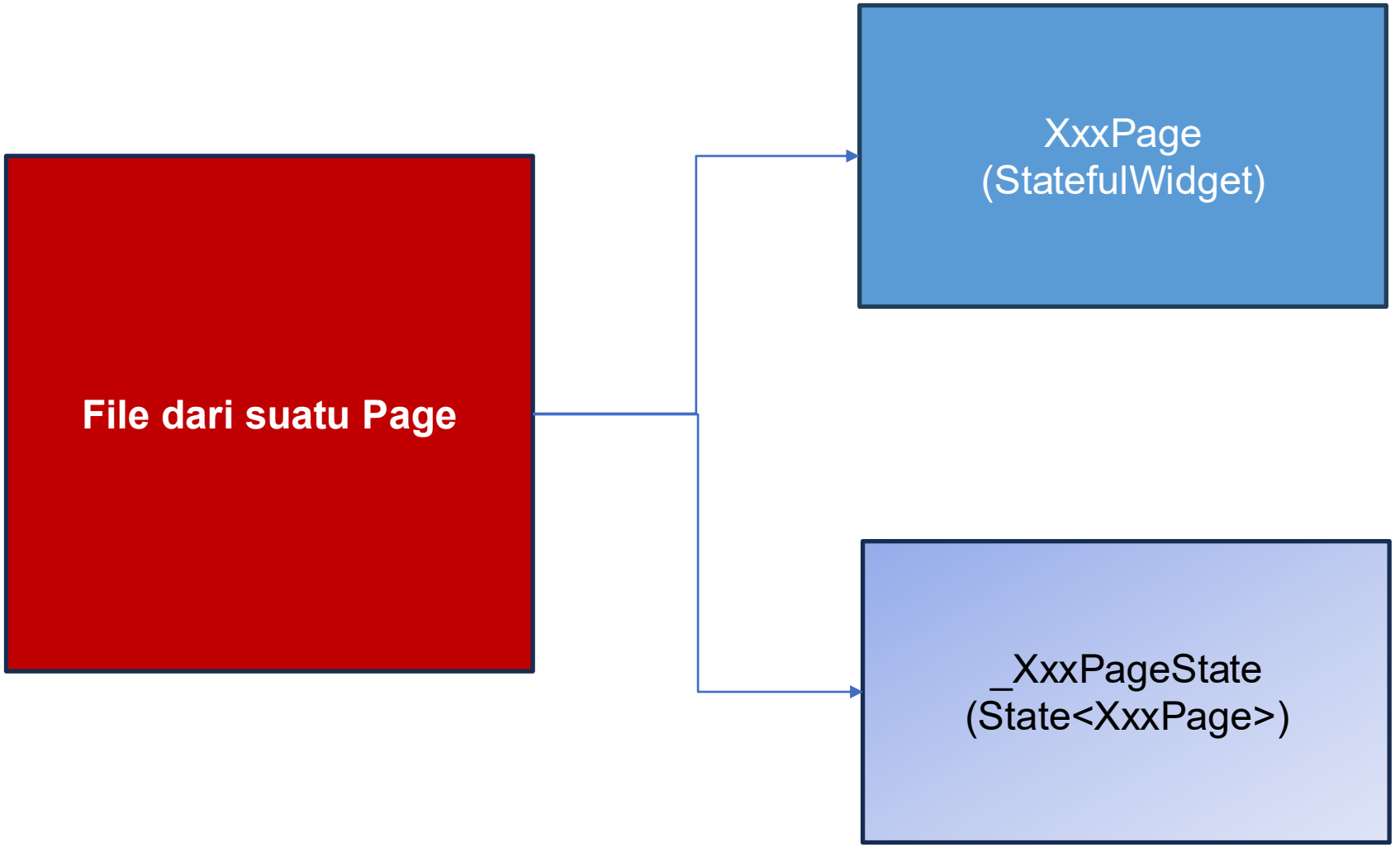
# 1. Widget Stack Dasar

- Pada dasarnya, sebuah aplikasi Flutter Adalah **tumpukan widget.**
- Proyek Flutter yang merupakan aplikasi Dart akan selalu diawali dari **void main()**;
- Dari main() akan dipanggil method **runApp(),** ini adalah method dari Flutter.
- Parameter runApp() adalah **root widget**.
  - Berupa suatu class custom yang Meng-extend class Widget.
  - Biasanya Stateless Widget.
- Root widget akan memiliki property **home**. Dimana, pada home tersebut bisa kita isi widget yang akan bertindak sebagai halaman aplikasi atau **page**.
- Page sendiri, di atasnya ada widget berjenis **Scaffold**, yang menjadi kerangka dasar halaman. Widget ini memiliki properti appBar dan body.
  - Pada body inilah kita gambar elemen-elemen UI yang kita butuhkan.

# 1. Widget Stack Dasar
## Ilustrasi



# 1. Widget Stack Dasar
## Page



# *Topik-2:* Layout

# 2. Layout

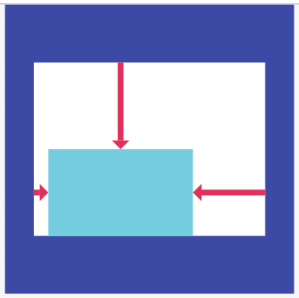
- Dalam suatu aplikasi, tentunya kita ingin agar tampilannya rapi.

- Agar tampilannya rapi, maka widget-widget yang ada dalam aplikasi kita, harus ditata sedemikian rupa:
  - Ditata dari kiri ke kanan, atau;
  - Dari atas ke bawah, atau;
  - Di tengah, atau;
  - Relatif antara komponen lainnya, atau yang lain sesuai selera.

- Untuk bisa secara otomatis menata widget yang diletakkan tanpa harus manual mengatur posisi sumbu x, y, Panjang, dan lebar, kita bisa memanfaatkan yang namanya **Layout**.

- Layout merupakan sekumpulan widget yang bisa memiliki **child** atau **children**, yang ketika kita isi widget lain, maka posisinya akan otomatis **mengikuti aturan** layoutnya.

# Jenis Layout – Single Child



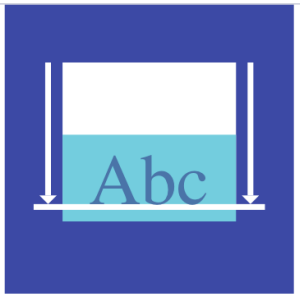
## Align

A widget that aligns its child within itself and optionally sizes itself based on the child's size.



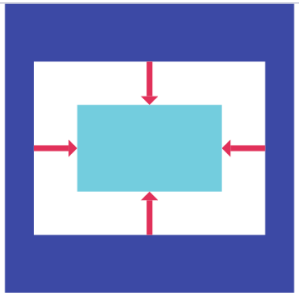
## AspectRatio

A widget that attempts to size the child to a specific aspect ratio.



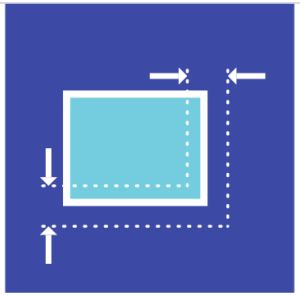

## Baseline

Container that positions its child according to the child's baseline.



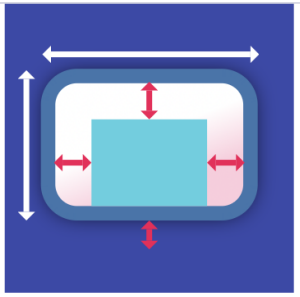
## Center

Alignment block that centers its child within itself.



## ConstrainedBox

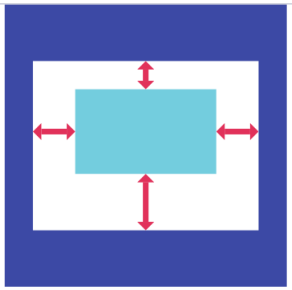
A widget that imposes additional constraints on its child.



## Container

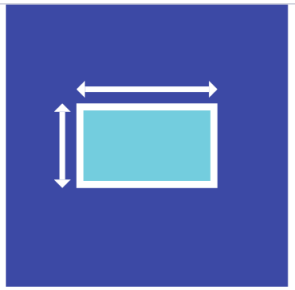
A convenience widget that combines common painting, positioning, and sizing widgets.

# Jenis Layout – Single Child



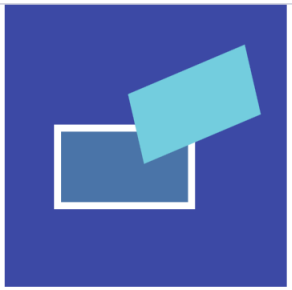
## Padding

A widget that insets its child by the given padding.



## SizedBox

A box with a specified size. If given a child, this widget forces its child to have a specific width and/or height (assuming values are...



## Transform

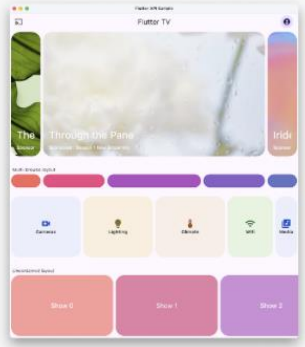
A widget that applies a transformation before painting its child.

# Jenis Layout – Multi Child



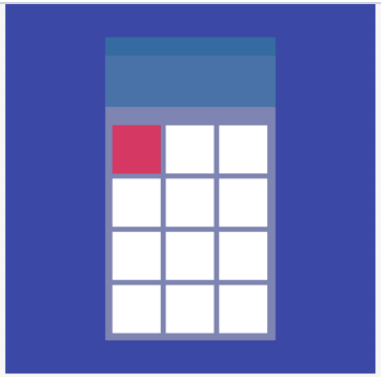
## Column

Layout a list of child widgets in the vertical direction.



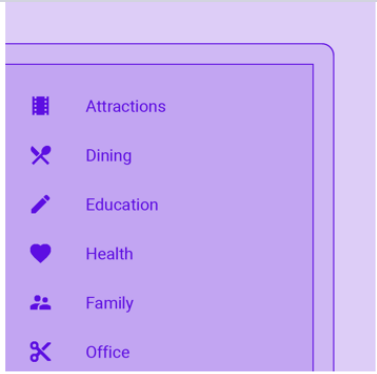
## CarouselView

A Material carousel widget that presents a scrollable list of items, each of which can dynamically change size based on the chosen layout.



## GridView

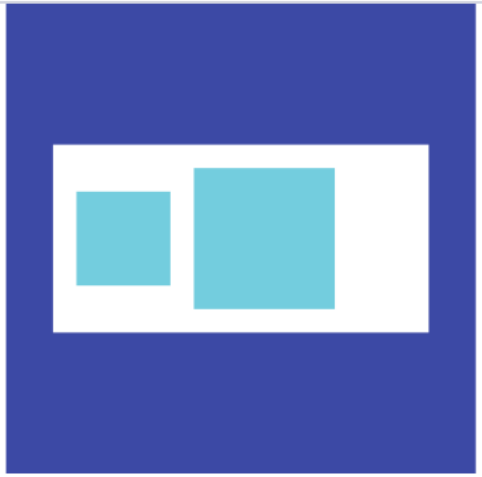
A grid list consists of a repeated pattern of cells arrayed in a vertical and horizontal layout. The GridView widget implements this component.




## ListView

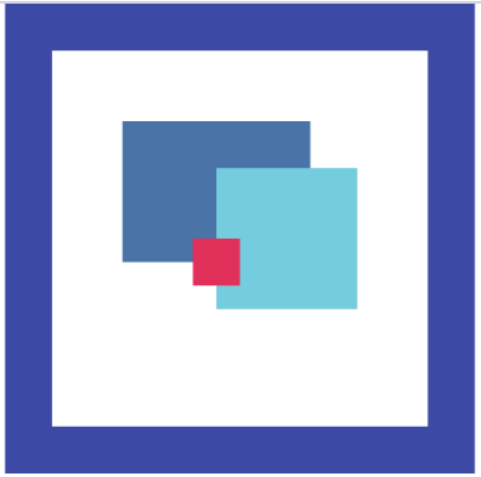
A scrollable, linear list of widgets. ListView is the most commonly used scrolling widget. It displays its children one after another in the scroll direction....

# Jenis Layout – Multi Child



## Row

Layout a list of child widgets in the horizontal direction.



## Stack

This class is useful if you want to overlap several children in a simple way, for example having some text and an image, overlaid with...

# Jenis Layout

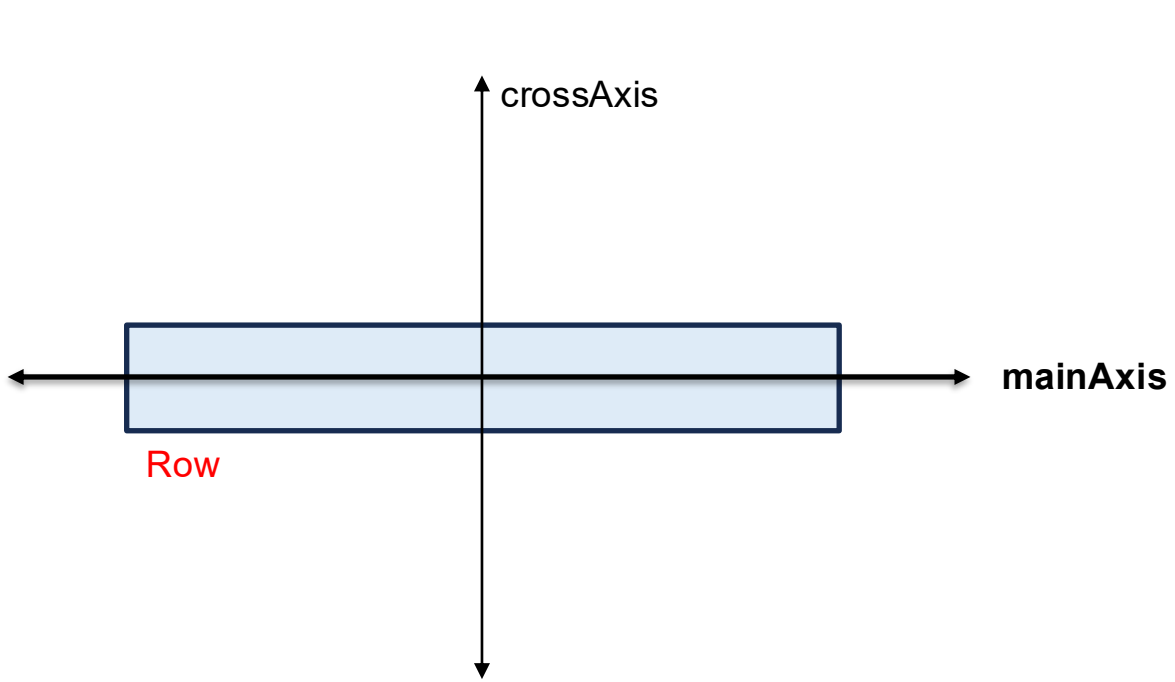
- Flutter menyediakan masih banyak jenis-jenis layout yang lainnya.

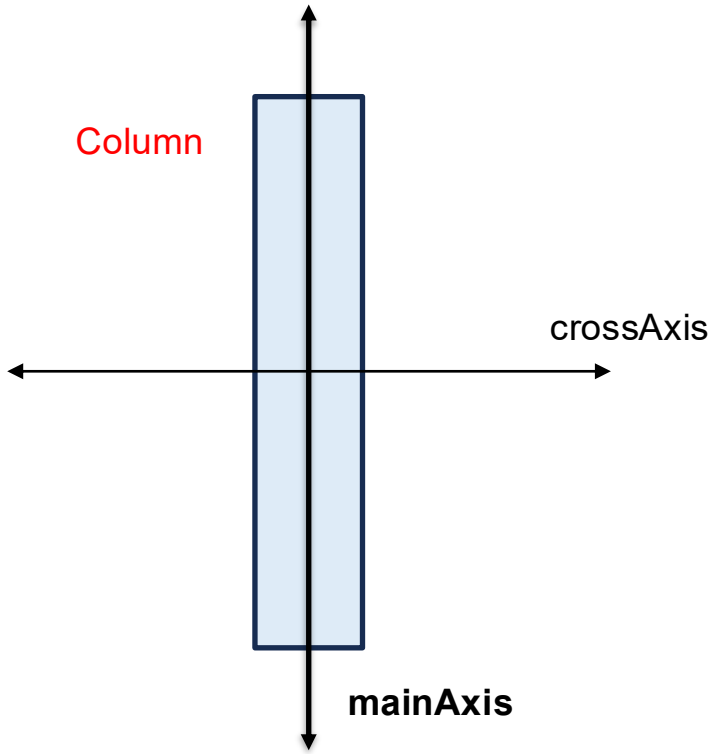
- Untuk memahami Layout selain yang ada pada slide ini, silahkan mengunjungi halaman dokumentasi Flutter tentang layout. [1]

# Alignment

- Pada Widget Layout biasanya child/children yang ada didalamnya bisa diatur agar rata kiri, kanan, tengah, atas, atau bawah.

- Untuk mengatur perataan tersebut, digunakan property **xxxAxisAlignment:**
  - **mainAxisAlignment**, → Sumbu yang **sejajar arah peletakan** child/children pada layout.
  - **crosAxisAlignment** → Sumbu yang berpotongan tegak lurus dengan mainAxis.



# *Topik-3:* Assets

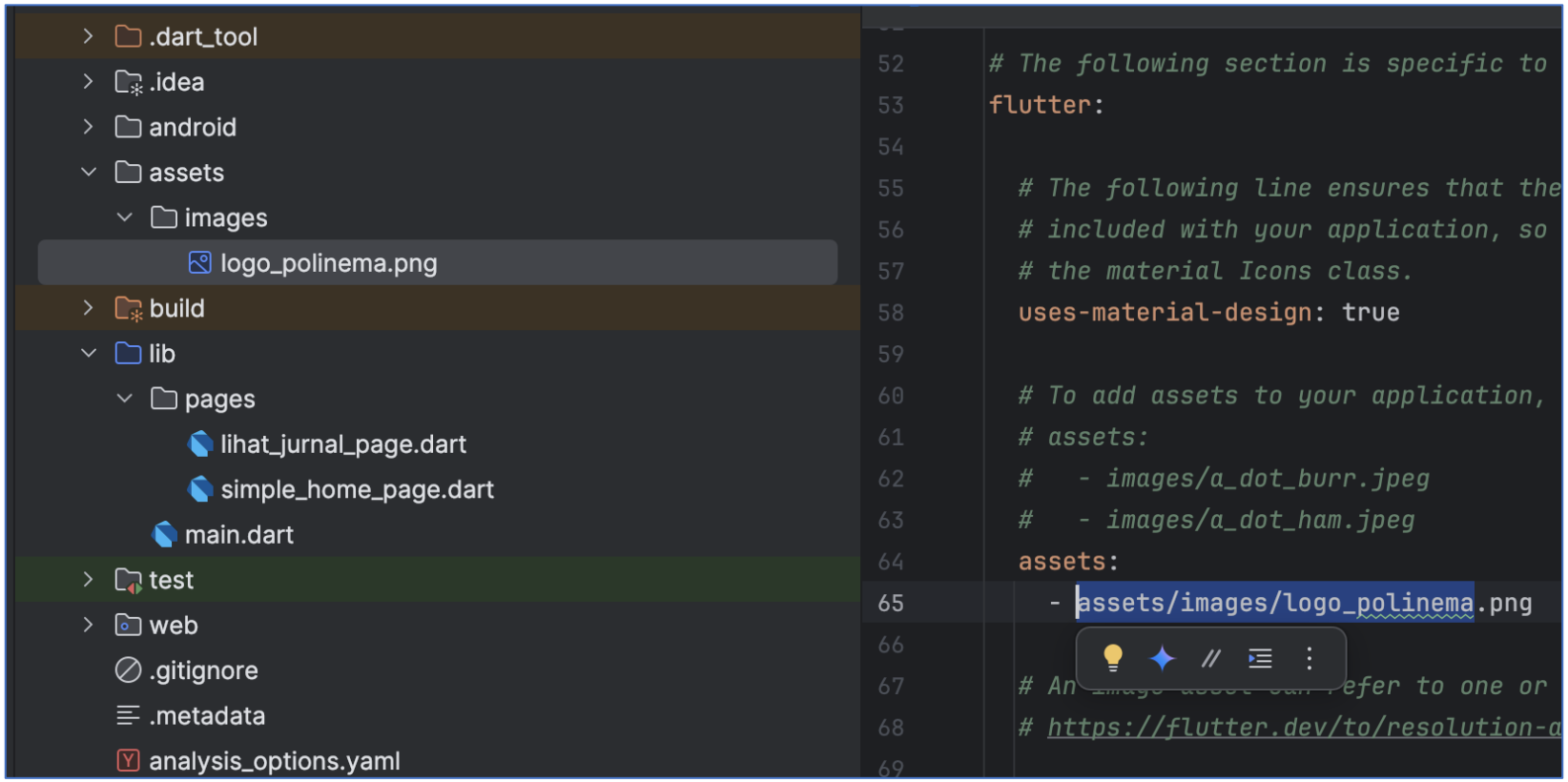
# 3. Assets

- Agar aplikasi kita lebih menarik, biasanya kita ingin menambahkan:
  - Gambar
  - Foto
  - Logo
  - Suara
  - File lain

- File-file statis yang ditambahkan ke dalam aplikasi kita disebut dengan **assets**.

- Untuk menambahkan asset kedalam aplikasi:
  - Buat folder untuk menampung asset
  - Registrasikan asset tersebut pada file **pubspec.yaml**
  - Full restart aplikasi Anda jika waktu di hot reload tidak ditemukan.

# 3. Assets
# Menambahkan Asset




- Posisi asset harus sesuai. Lalu tambahkan di file pubspec.yaml.
- File YAML adalah format yang **sensitive terhadap spasi**. Spasi yang digunakan adalah 2 spasi.

# 3. Assets
# Memanggil Aset Image

```dart
@override
Widget build(BuildContext context)
{
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    ), AppBar
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Image.asset('assets/images/logo_polinema.png'),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              ElevatedButton(onPressed: () => {_openLihatJurnalPage(context)}, child: Text('Lihat Jurnal')),
              SizedBox(width: 20),
              ElevatedButton(onPressed: () => {}, child: Text('Buat Jurnal'))
            ]
          ) Row
        ],
      ), Column
    ), Center
  ); Scaffold
}
```



# *Topik-4:* Navigasi

# 3. Navigasi

- Aplikasi mobile biasanya tidak hanya terdiri dari 1 halaman saja. Setidaknya 2 halaman:
  - Halaman login;
  - Halaman utama.

- Untuk berpindah halaman, kita menggunakan suatu class di dalam Flutter yang bernama **Navigator**.

- Navigator memiliki method static bernama **push()** dimana salah satu parameternya Adalah **router**.

- Pada **router** ini terdapat parameter **builder, dimana** kita inputkan halaman baru yang merupakan **tujuan** perpindahannya.

- Builder tadi merupakan parameter yang menerima **input berupa fungsi.** Sehingga kita menggunakan sintaksis fat arrow notation () =>.

# Menggunakan Navigator.push()

```
// Method untuk pindah ke halaman Lihat Jurnal
void _openLihatJurnalPage(BuildContext context)
{
  Navigator.push(
    context,
    MaterialPageRoute(
        builder: (context) => LihatJurnalPage(
            title: 'Lihat Jurnal')
    )
  );
}
```

# *Topik-5:* Tampilan Awal Aplikasi MyDJ

# 5. Tampilan Awal Aplikasi MyDJ

- Penjelasan topik ini ada di **Petunjuk Praktikum**.

# Pertanyaan?



# Terima Kasih

# Tugas

- Selesaikan langkah-langkah pada petunjuk praktikum.

# Referensi

1. https://docs.flutter.dev/ui/layout